

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030027

Name: Venkata Sai Krishna

SEC-8

PRACTICAL-5:

Aim: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

PROCEDURE :

1.Producer-Consumer :

```
venkatsaikrishna5@VSK-HP:~$ mkdir practical5
venkatsaikrishna5@VSK-HP:~$ cd practical5
venkatsaikrishna5@VSK-HP:~/practical5$ nano producer.c
venkatsaikrishna5@VSK-HP:~/practical5$ gcc producer.c -o producer
venkatsaikrishna5@VSK-HP:~/practical5$ ./producer
Child received: Data produced by parent process
Communication time: 0.000566814 seconds
```

2. `ls -l | grep ".c"` :

```
venkatsaikrishna5@VSK-HP:~/practical5$ nano pipegrep.c
venkatsaikrishna5@VSK-HP:~/practical5$ gcc pipegrep.c -o pipegrep
venkatsaikrishna5@VSK-HP:~/practical5$ ./pipegrep
-rw-r--r-- 1 venkatsaikrishna5 venkatsaikrishna5 688 Aug 29 0
9:49 pipegrep.c
-rwxr-xr-x 1 venkatsaikrishna5 venkatsaikrishna5 16432 Aug 29 0
9:48 producer
-rw-r--r-- 1 venkatsaikrishna5 venkatsaikrishna5 1055 Aug 29 0
9:48 producer.c
```

Code:

```
GNU nano 8.7.1 producer.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    int fd[2];
    pid_t pid;
    char buffer[100];
    struct timespec start, end;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid > 0)
    {
        close(fd[0]);

        strcpy(buffer, "Data produced by parent process");
    }
}
```

```
    write(fd[1], buffer, strlen(buffer) + 1);

    close(fd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time = (end.tv_sec - start.tv_sec) +
                   (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Communication time: %.9f seconds\n", time);
}
else
{
    close(fd[1]);

    read(fd[0], buffer, sizeof(buffer));

    printf("Child received: %s\n", buffer);

    close(fd[0]);

    exit(0);
}

return 0;
}
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int fd[2];
    pid_t pid;
    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }
    if (pid > 0)
    {
        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
        perror("execlp");
        exit(1);
    }
    else
    {
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execlp("grep", "grep", ".c", NULL);

        perror("execlp");
        exit(1);
    }
    return 0;
}
```

Observation :

1. The parent process successfully produced/wrote data into the anonymous pipe.
2. The child process successfully received/read the data from the pipe.
3. The communication time was measured successfully.
4. In the second program, `ls -l` output was successfully passed through the pipe to `grep ".c"`.
5. Only files containing `.c` in their listing were displayed.

Result :

The producer-consumer communication system was successfully implemented using an anonymous pipe, and its communication efficiency was measured.

The command `ls -l | grep ".c"` was successfully implemented using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.